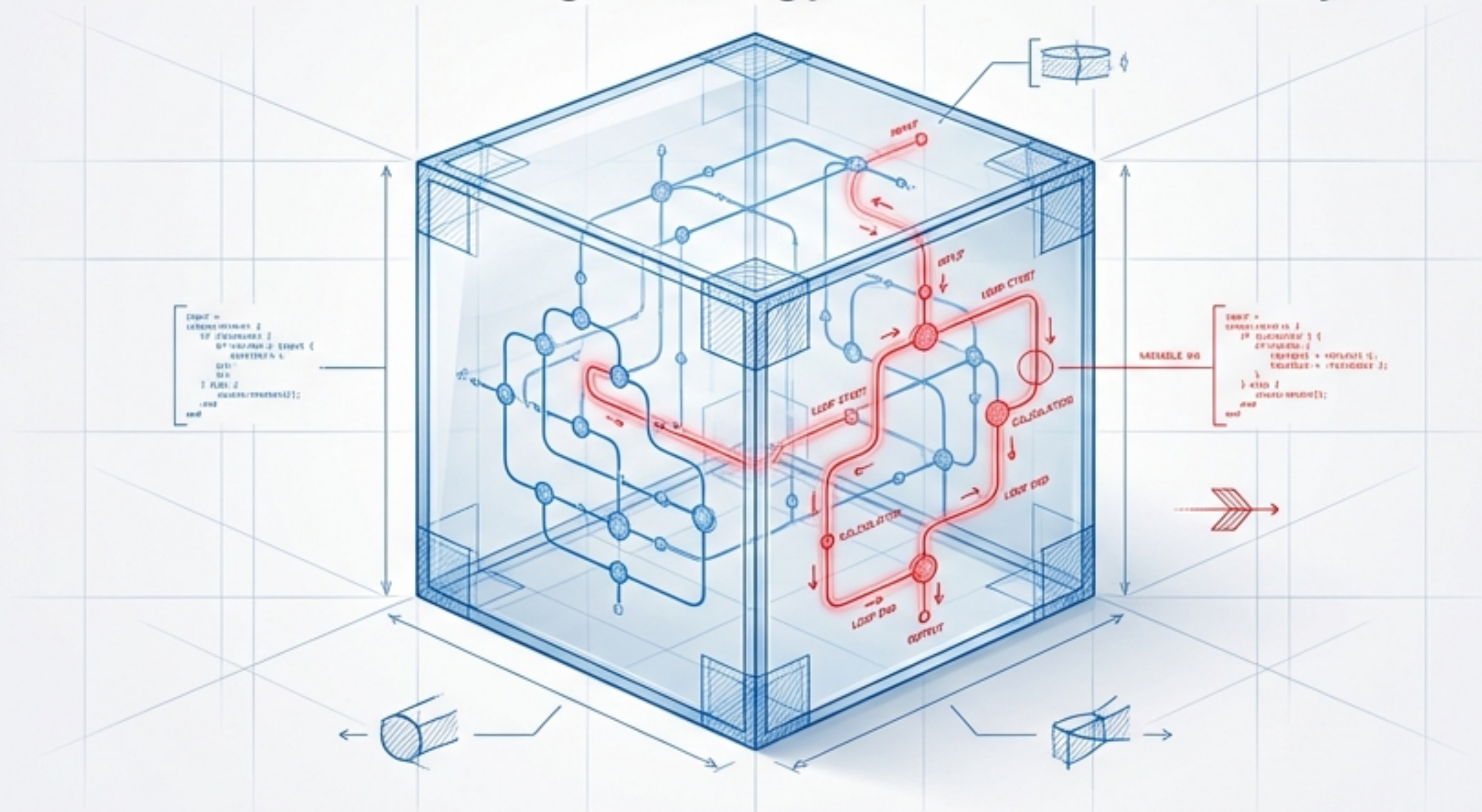


Chiến Lược Kiểm Thử Hộp Trắng & Phân Tích Luồng Dữ Liệu

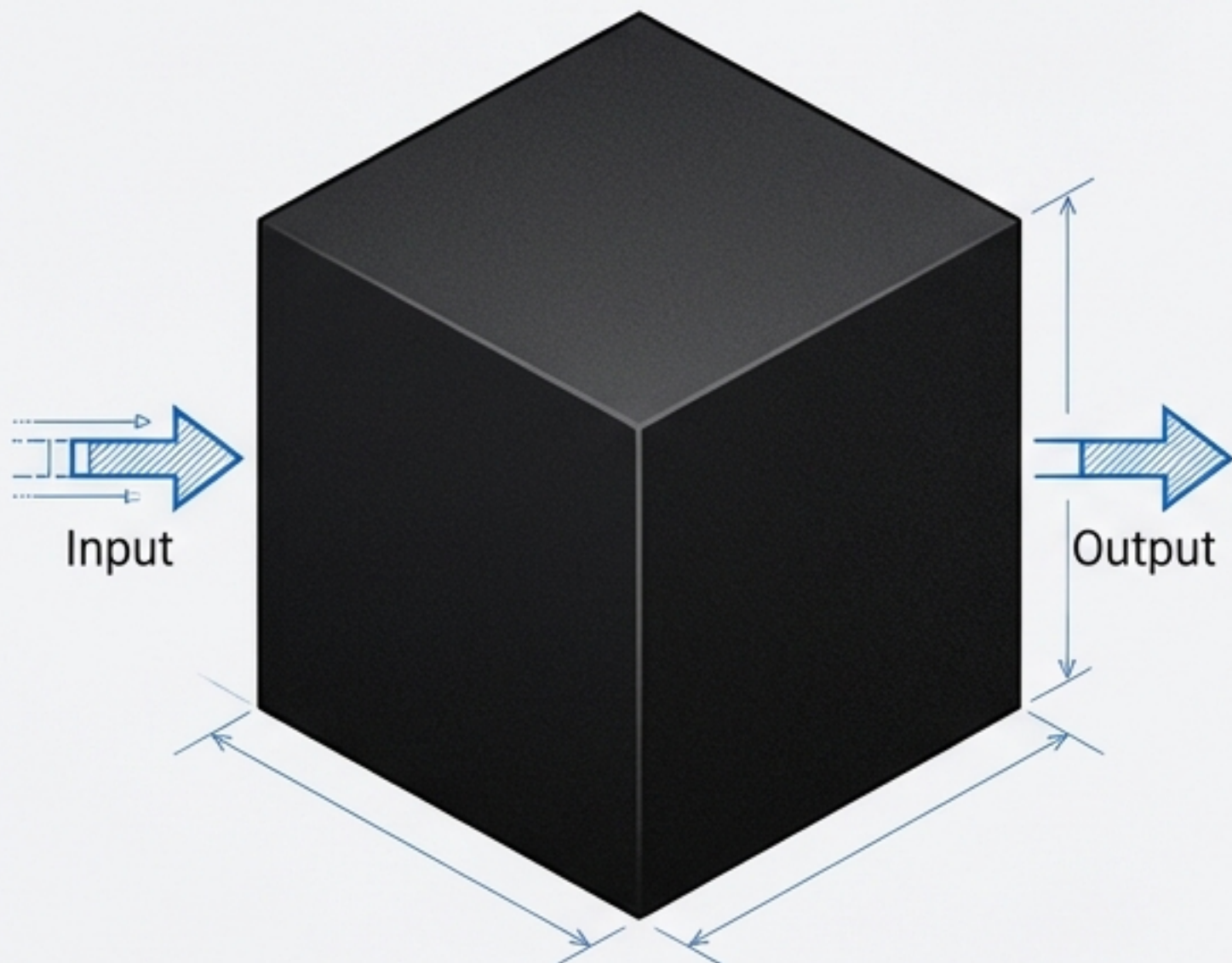
White Box Testing Strategy & Data Flow Analysis



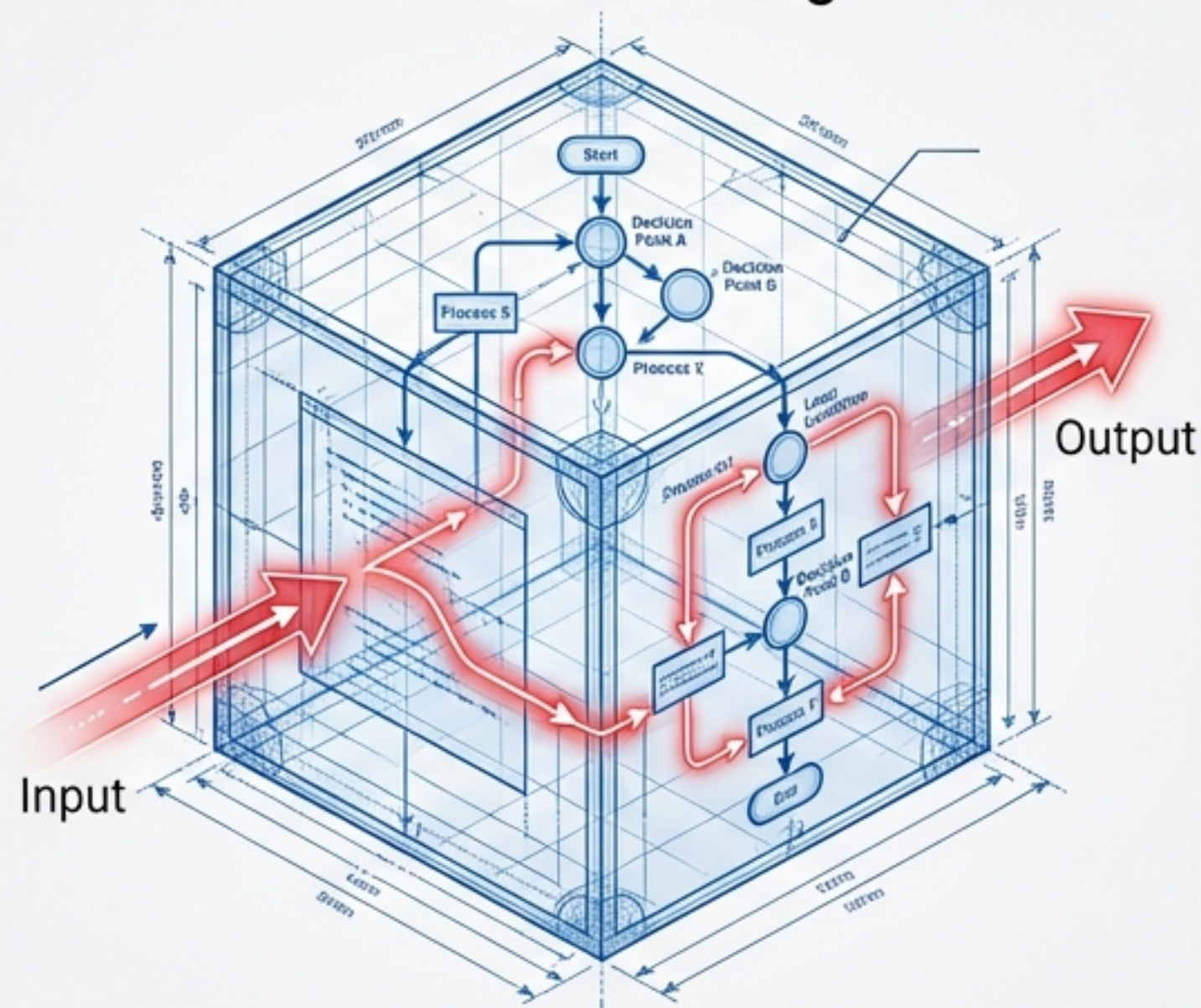
Tiếp cận giải phẫu: Từ cấu trúc điều khiển đến vòng đời biến số.

Tổng Quan: Kiểm Thử Hộp Trắng (Glass Box Testing)

Black Box Testing

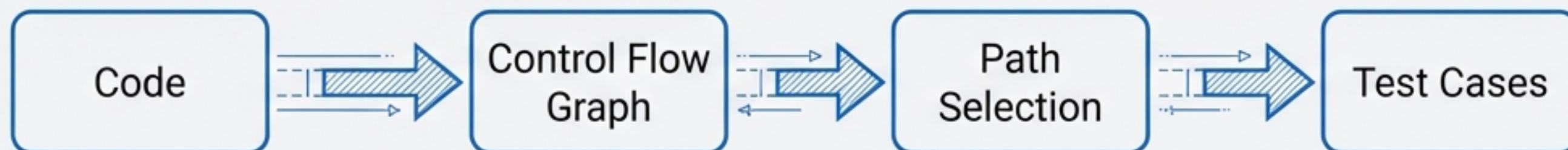


White Box Testing

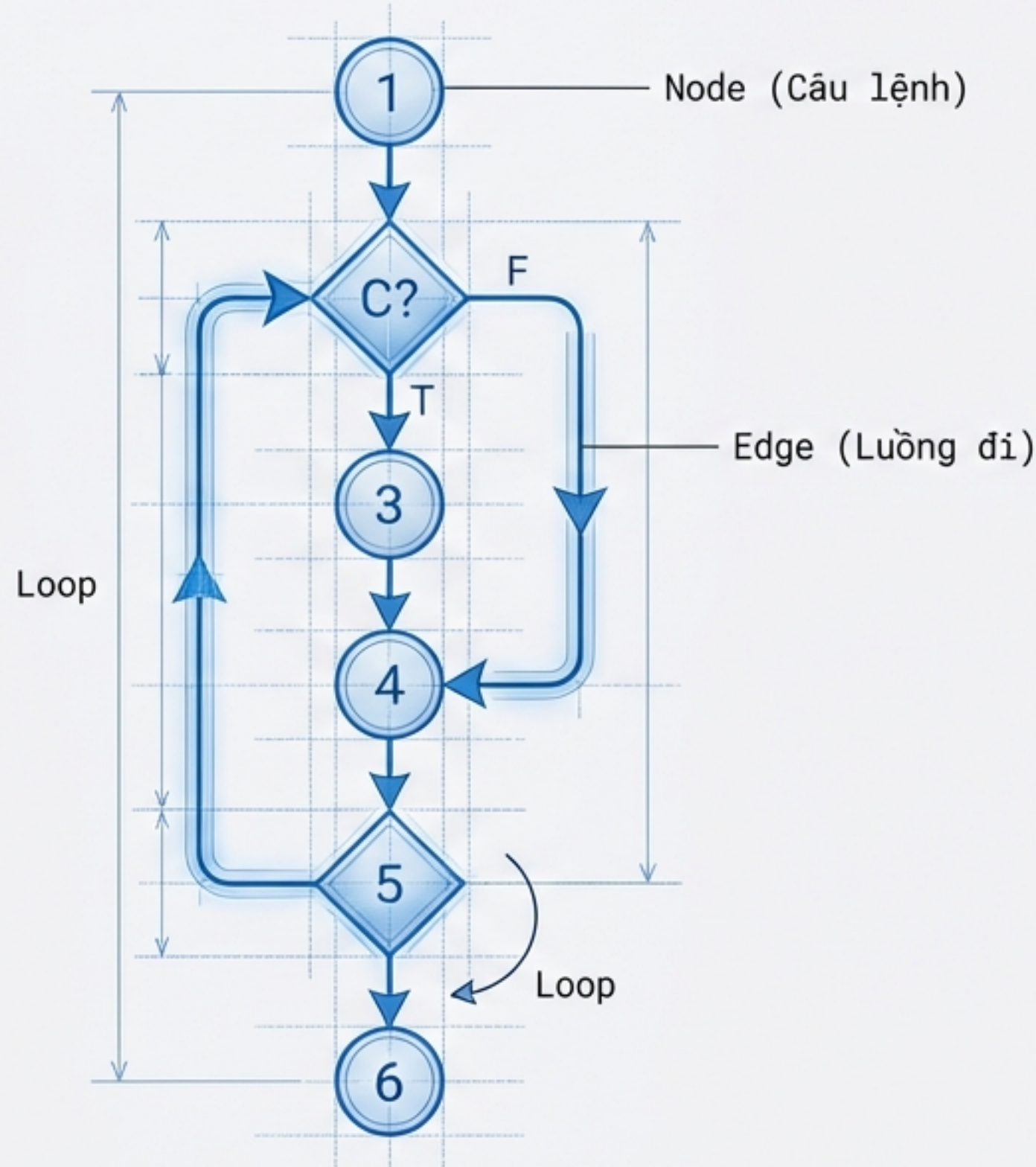


Đặc điểm chính:

- Truy cập trực tiếp mã nguồn (Source Code).
- Tập trung vào logic nội tại và luồng đi.
- Áp dụng chủ yếu ở mức Unit Testing (Kiểm thử đơn vị).



Bộ Khung: Kiểm Thử Luồng Điều Khiển (Control Flow)



Mô hình đồ thị (CFG)

Biểu diễn mã nguồn dưới dạng các đỉnh (Nodes) và cạnh (Edges).

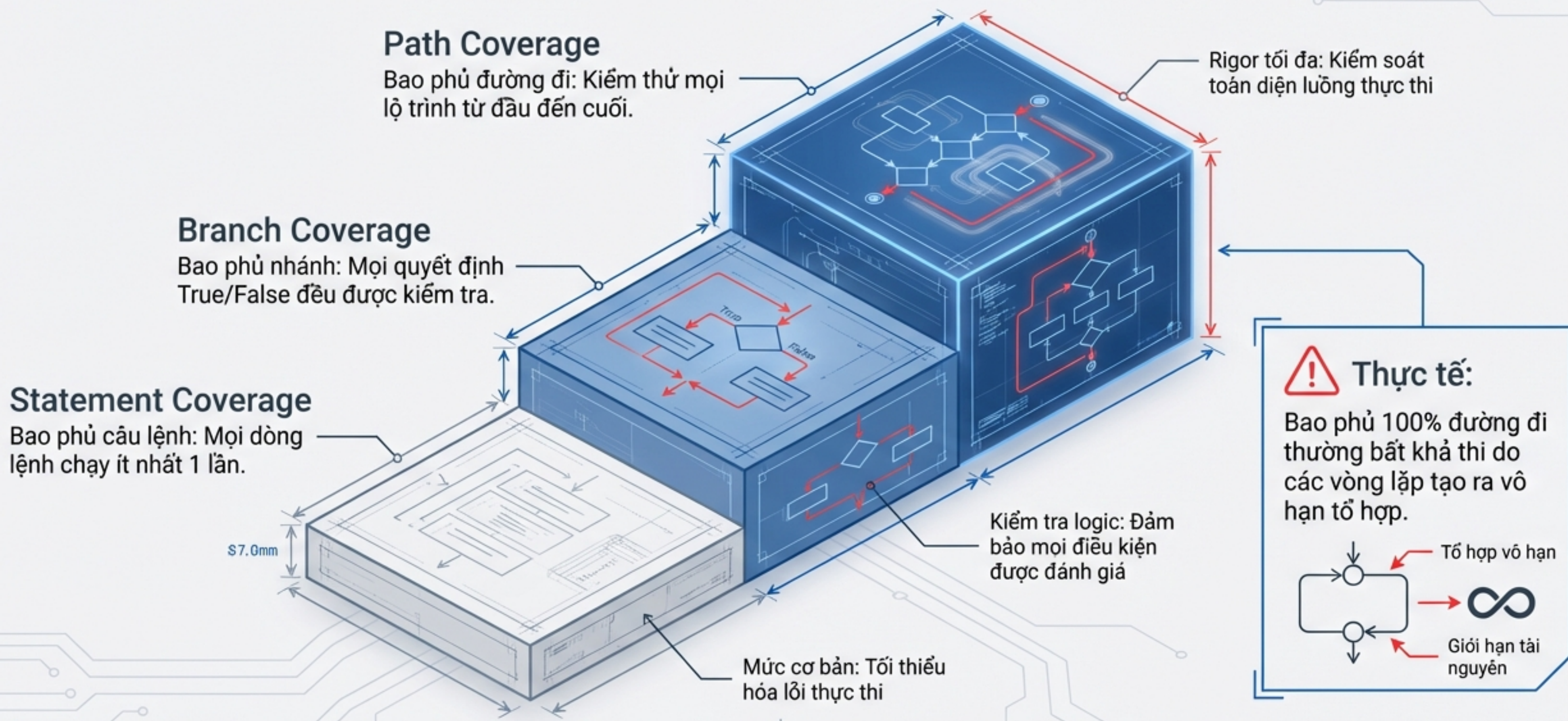
Độ phức tạp Cyclomatic ($V(G)$)

$$V(G) = E - N + 2$$

Xác định số lượng đường cơ sở (basis paths) tối thiểu cần kiểm thử.



Các Cấp Độ Bao Phủ Luồng Điều Khiển



Tại Sao Cần Kiểm Thử Luồng Dữ Liệu? (Data Flow)

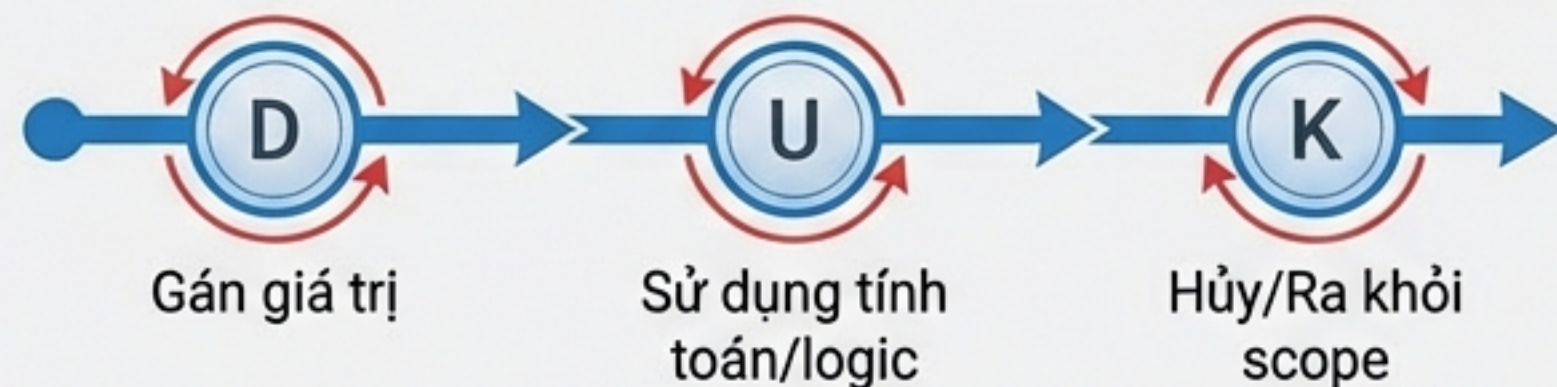
```
1 int main() {  
2   int data_var; // Definition  
3   // ... some operations ...  
4   if (condition) {  
5     data_var = calculate_value();  
6   }  
7   process(data_var); // Usage  
8   return 0;  
9 }
```

Vấn đề:

Luồng điều khiển (Control Flow) có thể đúng, nhưng dữ liệu có thể bị xử lý sai.

Ví dụ: Biến được khai báo nhưng chưa khởi tạo giá trị, hoặc bị gán đè trước khi sử dụng.

Giải pháp - Vòng đời biến số:

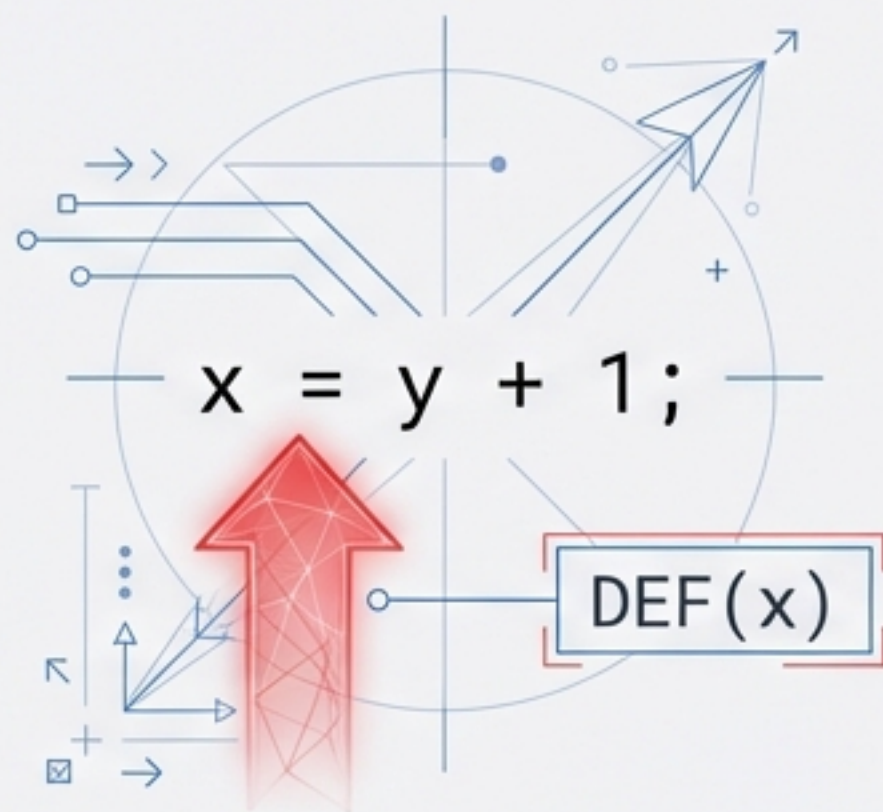


Key Insight:

Mục tiêu: Theo dõi hành trình từ nơi biến sinh ra (Def) đến nơi biến được dùng (Use).

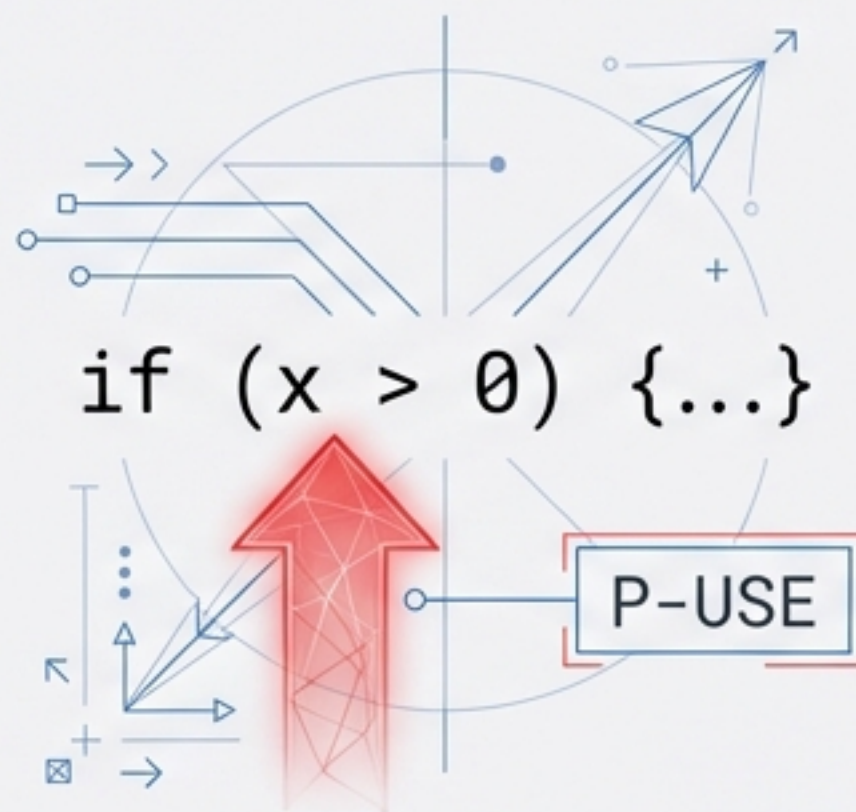
Giải Phẫu Dữ Liệu: Các Định Nghĩa Cốt Lõi

DEF (Định nghĩa)



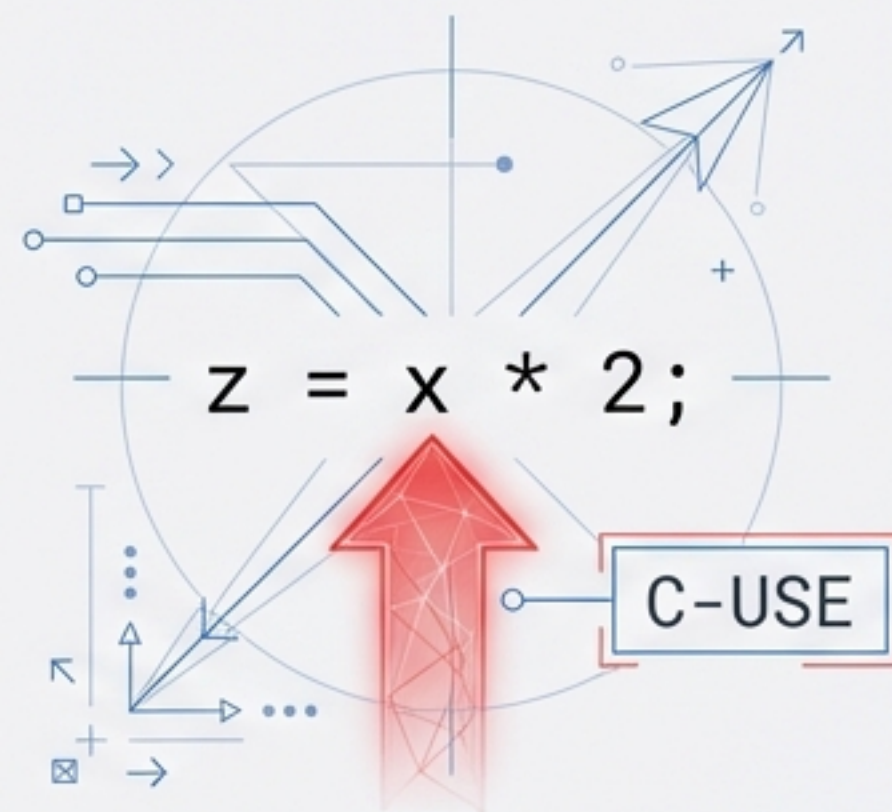
Nơi biến nhận giá trị mới.

P-USE (Dùng vị từ)



Predicate Use. Biến quyết định luồng đi (trong nhánh if, while).

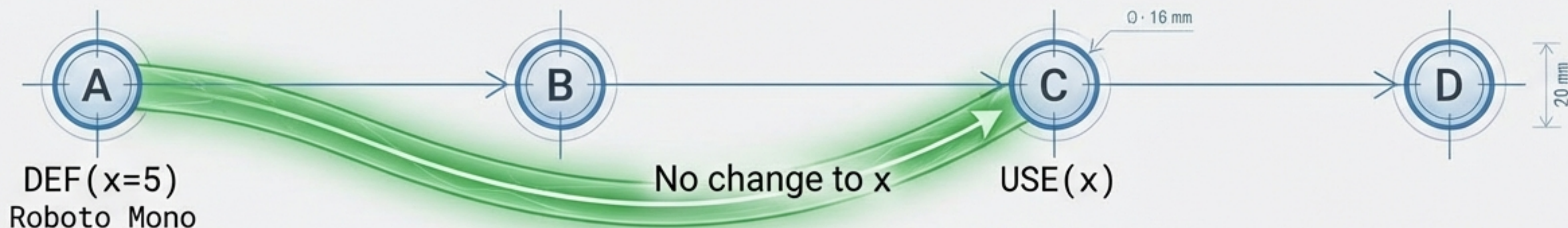
C-USE (Dùng tính toán)



Computation Use. Biến tham gia tính toán hoặc lệnh in.

Đường Đi DU & Đường Đi Sạch (Def-Clear)

Scenario 1: Def-Clear Path



Def-Clear Path: Giá trị $x=5$ tồn tại an toàn đến khi sử dụng.

Scenario 2: Not Def-Clear Path



Blocked Path: Giá trị ban đầu bị ghi đè tại B. Đường đi không sạch.

Các Tiêu Chuẩn Bao Phủ Dữ Liệu

All-Defs
Roboto Mono

Mỗi định nghĩa (Def) đến ít nhất 1 điểm sử dụng (Use).

All-Uses

Mỗi định nghĩa đến TẤT CẢ các điểm sử dụng (C-use & P-use).

Bao gồm: All-P-Uses (nhánh) & All-C-Uses (tính toán).

All-Uses

Mỗi định nghĩa đến TẤT CẢ các điểm sử dụng (C-use & P-use).

Bao gồm: All-P-Uses (nhánh) & All-C-Uses (tính toán).

All-DU-Paths

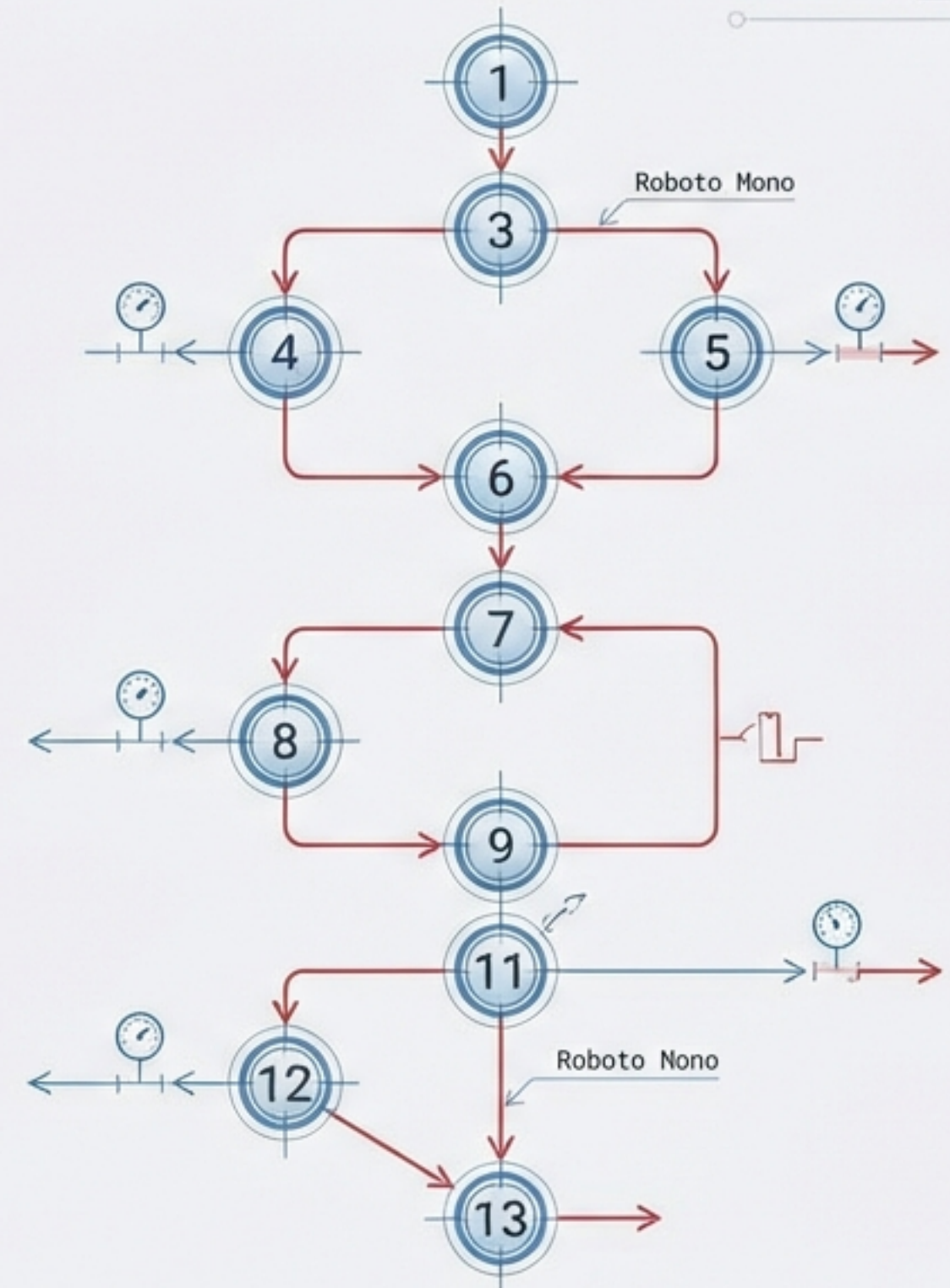
Mạnh nhất: Bao phủ mọi đường đi đơn giản từ Def đến Use.

All-DU-Paths

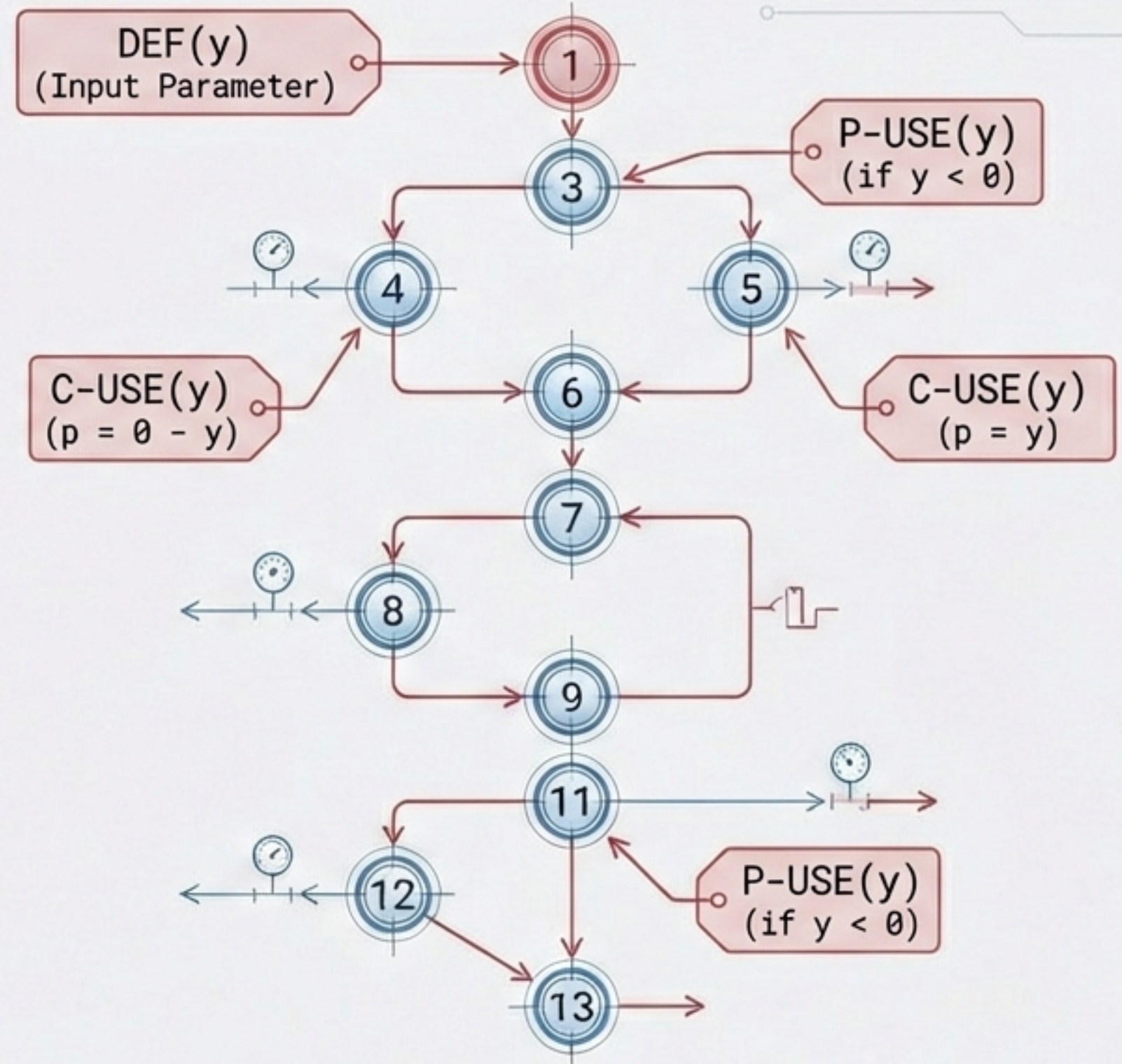
Mạnh nhất: Bao phủ mọi đường đi đơn giản từ Def đến Use.

Case Study: Hàm pow(x,y)

```
1. void pow (int x, y) {  
2.     float z; int p;  
3.     if (y < 0)  
4.         p = 0 - y;  
5.     else p = y;  
6.     z = 1.0;  
7.     while (p != 0) {  
8.         z = z * x;  
9.         p = p - 1;  
10.    }  
11.    if (y < 0)  
12.        z = 1.0 / z;  
13.    printf(z);  
14. }
```



Phân Tích Biến 'y' trong pow(x,y)



Mục tiêu

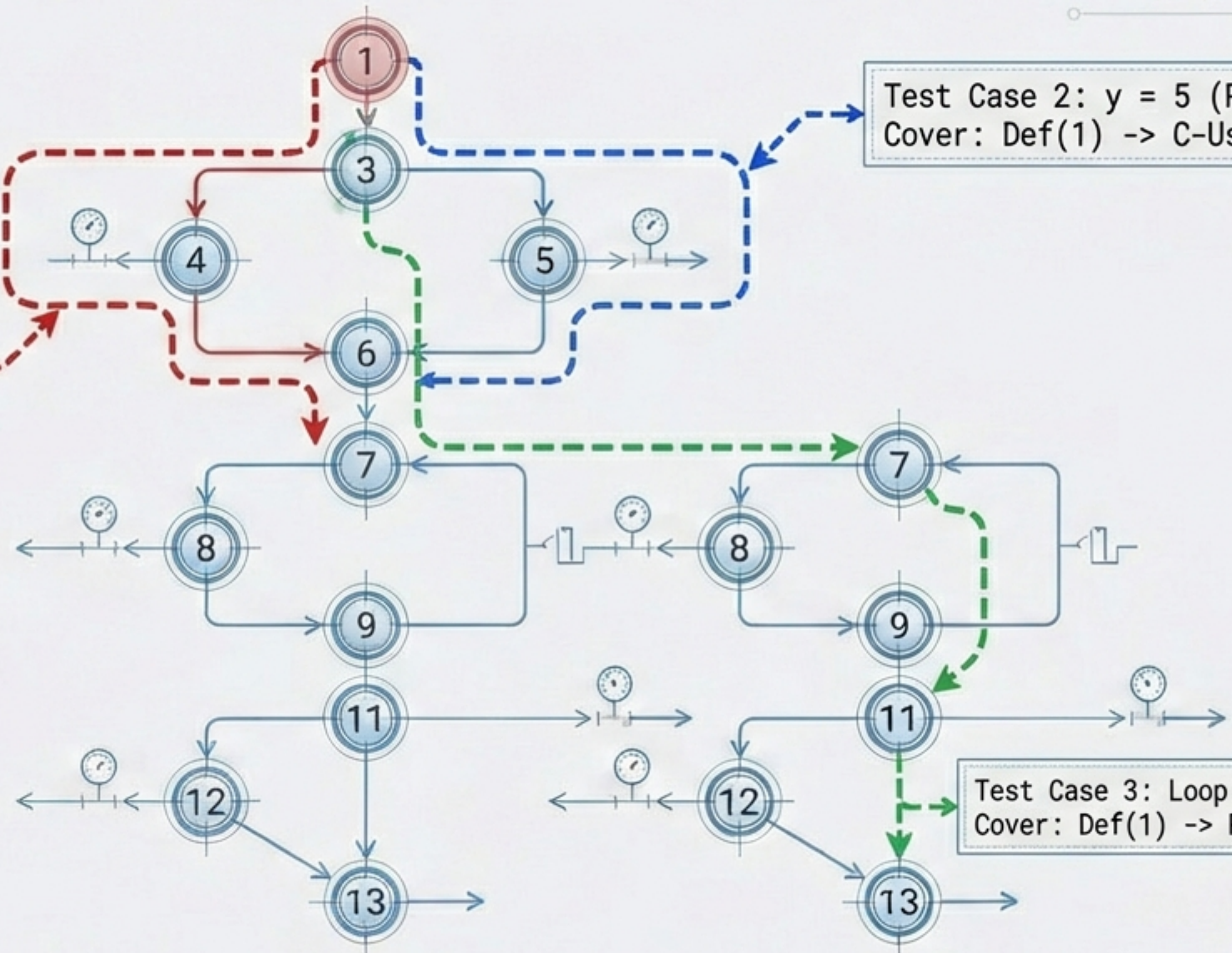
Tìm đường đi từ Node 1 đến các Node 3, 4, 5, 11.

Xác Định Đường Đi DU (DU-Paths) cho biến 'y'

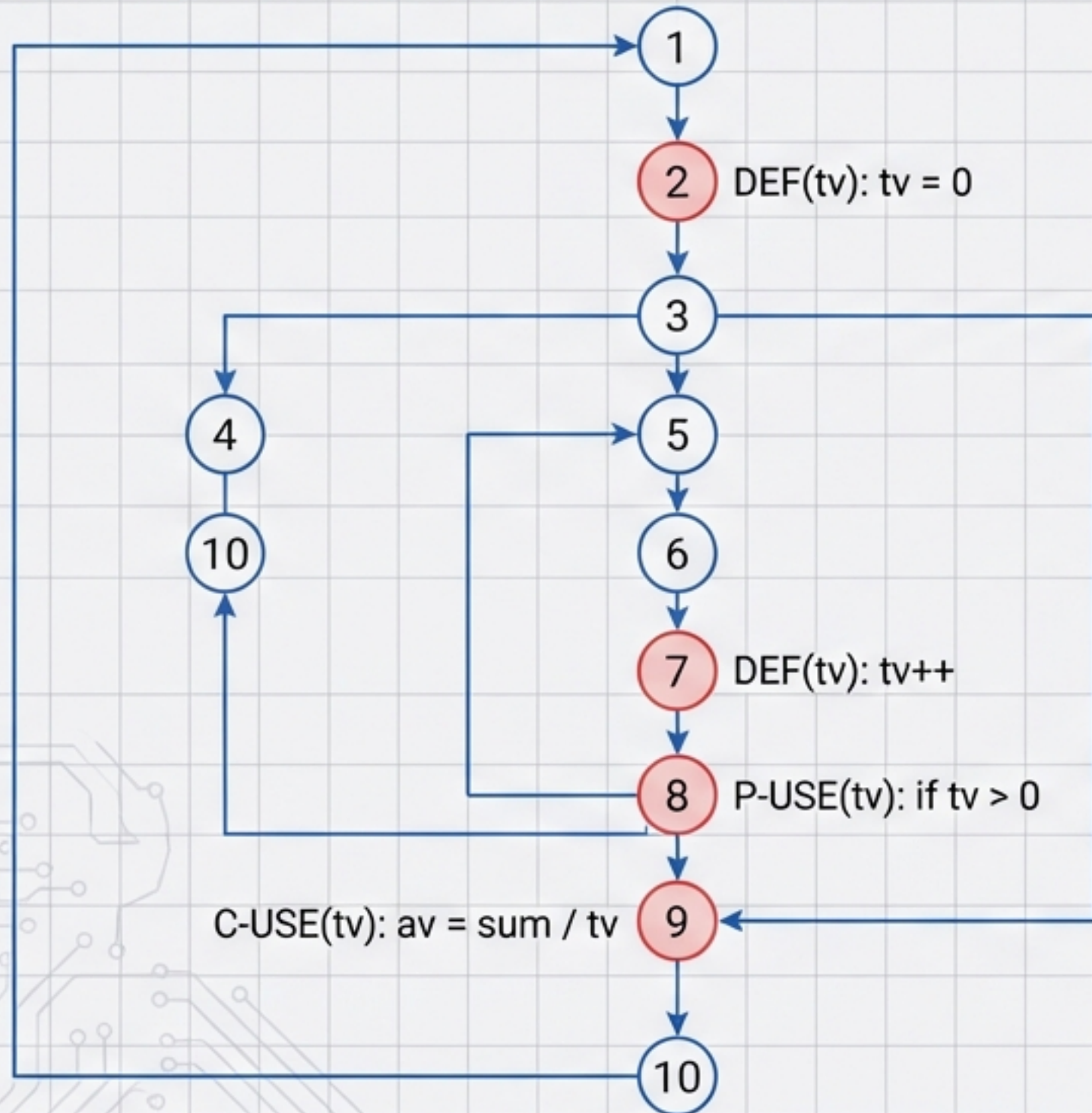
Test Case 1: $y = -5$ (Negative)
Cover: Def(1) $\rightarrow$ C-Use(4)

Test Case 2: $y = 5$ (Positive)
Cover: Def(1) $\rightarrow$ C-Use(5)

Test Case 3: Loop Execution
Cover: Def(1) $\rightarrow$ P-Use(11)



Ví dụ Mở Rộng: Hàm ReturnAverage



Focus Box

Biến quan tâm: tv (Total Valid count)

DEF: Node 2 ($tv = 0$)

DEF (in loop): Node 7 ($tv++$)

P-USE: Node 8 ($\text{if } tv > 0$)

C-USE: Node 9 ($av = \text{sum} / tv$)



Critical Insight: Cần test case sao cho 'tv' vẫn bằng 0 (không vào loop hoặc không thỏa điều kiện) để kiểm tra lỗi chia cho 0 tại Node 9.

Công Cụ Quản Lý: Ma Trận Kiểm Thử

	1	2	3	4	5	6	7	8	9	10	11	
1	1	1	1	1								$V(G)=4$
2	2	1	1	1								$1+1-1=1$
3	4	1	1	1		1						$1+1-1=1$
4	5	1	1		10		1					$1-1=0$
6						6	7					$1+1-1=1$
7							6	8				$1-1=0$
8								7	9			$1-1=0$
9								1	8	9		$1-1=0$
10										1		$1-1=0$
11		1										$\Sigma+1=3+1=4$

- Biến đổi đồ thị (Graph) thành Đại số (Matrix).
- Hỗ trợ tính toán tự động các đường cơ sở và độ phức tạp.

Tổng Kết & Đánh Giá Chiến Lược



Ưu Điểm

- Phát hiện lỗi logic tinh vi.
- Tìm ra mã thừa (dead code).
- Độ tin cậy cao cho thuật toán.



Nhược Điểm

- Chi phí cao, tốn thời gian.
- Yêu cầu chuyên môn sâu về code.
- Khó áp dụng diện rộng.



Khuyến Dùng

- Unit Testing cho module hạt nhân.
- Hệ thống an toàn (Safety-critical).
- Kết hợp với Black Box.

Tài Liệu Tham Khảo & Nguồn Dữ Liệu



1. 3.3 Data Flow Testing VN (Scribd)



2. White Box Testing Techniques (Scribd)



3. Thiết kế Test-Case trong Kiểm thử Phần mềm (Luận văn Thạc sĩ - Phạm Thị Hùy)



4. Tìm hiểu về kiểm thử hộp trắng (Viblo - Nguyen Thi Thuy)

Kiểm thử không chỉ là tìm lỗi, mà là thấu hiểu dòng chảy của dữ liệu để kiến tạo phần mềm bền vững.